Entorno Cliente · DWEC

Guía de examen DWEC — Unidades 1–2

Esto no es un resumen para leer una vez: es un cuaderno de trabajo para los días previos al examen. La Unidad 1 va condensada, con lo que se pregunta y no con la historia completa. La Unidad 2 va en el único formato que sirve para un examen de código: un fragmento corto, la salida real y la razón de por qué sale eso.

Cómo está montada

- Cada apartado de la Unidad 2 lleva ejemplos con el código, la salida y dos o tres viñetas. La viñeta marcada como **Trampa** señala el error que se cobra en el examen.
- Todas las salidas están comprobadas ejecutando el código con Node 24. El resultado es el mismo que en el navegador salvo cuando se toca el DOM, `prompt`, `alert` o `window`, y ahí se avisa.
- Los bloques de salida reproducen lo que escribe `console.log`. Los arrays se muestran como `[4, 8, 9]`, formato de la consola, y no como `[ 4, 8, 9 ]`, que es como los imprime Node por terminal. En DevTools los arrays llevan delante su longitud: `(3) [4, 8, 9]`.
- Las cadenas se muestran sin comillas, igual que en la consola. Cuando el tipo importa (una cadena `"52"` frente al número `52`), lo dice la viñeta.
- Los errores se citan con el mensaje exacto del motor. Si piden "qué error da", el nombre (`SyntaxError`, `TypeError`, `ReferenceError`) es la mitad de la nota; el mensaje concreto, la otra mitad.
- Un detalle de entorno: `typeof window` devuelve `"undefined"` en Node y `"object"` en el navegador, porque allí `window` es el objeto global. Todo lo demás del temario se comporta igual.

Unidad 1 — Teoría condensada

1.1 Modelos de ejecución: cliente y servidor

Toda aplicación web usa el modelo cliente/servidor. El cliente es el equipo y el navegador de la persona que usa la web; el servidor guarda los datos y ejecuta la lógica que no puede estar en manos del usuario. Se comunican con peticiones HTTP o HTTPS.

Dimensión	Cliente (front-end)	Servidor (back-end)
Dónde se ejecuta	Navegador del usuario	Servidor remoto o nube
Código	Público: se ve con F12	Privado, nunca llega al usuario
Tecnologías	HTML, CSS, JavaScript, React, Vue, Angular	PHP, Python, Java, Node.js, C#

Dimensión	Cliente (front-end)	Servidor (back-end)
Acceso a datos	Indirecto, por HTTP	Directo a SQL o NoSQL
Recursos	CPU, RAM y batería del usuario	Potencia del servidor
Seguridad	Baja: es modificable	Alta: cobros, permisos, roles

Los tres pilares del front-end son HTML (estructura), CSS (presentación) y JavaScript (lógica y eventos).

Qué va en cada lado. En el cliente, lo inmediato: menús, modales, acordeones, cambios de estilo y ordenar o filtrar datos que ya están cargados en memoria. En el servidor, lo que no se puede confiar al usuario: cobros, comprobación de roles, consultas sobre millones de registros y cualquier validación con efecto real.

Web clásica frente a SPA. En la web clásica cada clic lanza una petición síncrona y el servidor devuelve un HTML nuevo entero: la pantalla parpadea y se queda en blanco un instante. En una SPA se descarga la plantilla una vez y después JavaScript pide solo datos en JSON y actualiza las partes del árbol que cambian, sin recargar.

- **Trampa:** validar solo en el cliente. El usuario abre la consola, cambia el `if` y sigue adelante. Toda validación con consecuencias se repite en el servidor.
- **Trampa:** poner una contraseña, una clave de API o la cadena de conexión en el JavaScript. Es código público: se lee con F12 y no hay forma de ocultarlo.
- **Trampa:** descargar un catálogo de ocho millones de registros a un array del navegador. La solución es paginación y filtrado en servidor.
- Que el servidor sea invisible no hace seguro al cliente: el back-end no es accesible, el front-end lo audita cualquiera.

1.2 El navegador por dentro

Un navegador se divide en siete módulos:

- **Interfaz de usuario.** La ventana: barra de direcciones, pestañas, botones.
- **Motor del navegador.** Intermediario entre la interfaz y los motores internos.
- **Motor de renderizado.** Lee HTML y CSS, calcula tamaño y posición de cada elemento y lo dibuja.
- **Motor de JavaScript.** Interpreta el código. Aplica JIT: detecta las funciones que se repiten y las compila a código máquina.
- **Capa de red.** Envía y recibe datos, resuelve DNS y valida certificados HTTPS.
- **UI Backend.** Conecta con el sistema operativo para dibujar ventanas con aspecto nativo.
- **Almacenamiento.** Cookies, caché y bases de datos locales.

El motor de renderizado y el de JavaScript están separados. Cuando JS cambia un texto, avisa al de renderizado para que recalcule y repinte.

Navegador	Motor de renderizado	Motor de JavaScript
Chrome, Edge, Brave, Opera	Blink	V8
Firefox	Gecko	SpiderMonkey
Safari	WebKit	JavaScriptCore

Chrome usó WebKit hasta 2013, cuando Google copió el proyecto y lo continuó como Blink. En iPhone y iPad cualquier navegador usa WebKit, aunque se llame Chrome o

Firefox, porque lo impone la tienda de Apple.

Del código a los píxeles, en cuatro pasos:

- **DOM y CSSOM.** El HTML se lee y se construye el árbol DOM en memoria; el CSS genera el árbol CSSOM.
- **Render Tree.** Se combinan los dos. Solo entran los elementos visibles: el `<head>` y lo que tiene `display: none` quedan fuera.
- **Layout.** Se calculan ancho, alto y coordenadas de cada caja.
- **Paint.** Se pintan los píxeles.

Un `<script>` detiene la lectura del documento mientras se descarga y se ejecuta. Si es pesado, la pantalla se queda en blanco.

Salida	Qué hace	¿La ve el usuario?
`console.log()`	Escribe en el panel Consola	No
`innerHTML`	Sustituye el marcado y el texto del nodo	Sí
`document.write()`	Escribe en el flujo de la página	Sí
`window.alert()`	Abre un diálogo modal nativo	Sí

- **Reflow:** el cambio altera dimensiones o posición (`fontSize`, `innerHTML`). El navegador recalcula el espacio y desplaza lo que rodea. Es lo caro.
- **Repaint:** el cambio es solo visual y no mueve nada (`color`, `backgroundColor`). Se repintan los píxeles afectados.
- **Trampa:** confundir la consola con la página. Si el enunciado pide que el usuario vea algo, `console.log()` no sirve.
- **Trampa:** llamar a `document.write()` con la página ya cargada. Borra el documento entero y deja solo lo escrito en esa llamada.
- **Trampa:** `alert()` es síncrono y bloqueante: congela el hilo principal y no atiende ningún evento hasta que se pulsa Aceptar.
- **Trampa:** `innerHTML` con datos del usuario abre la puerta al XSS. Para texto plano, `textContent`.
- **Trampa:** `style.font-size` no existe. El guion es una resta: se escribe `style.fontSize`, `style.backgroundColor`, `style.marginTop`.
- `getElementById()` devuelve `null` si el `id` no existe o el elemento aún no está en el DOM, y la línea siguiente lanza `TypeError: Cannot read properties of null`.

1.3 Lenguajes de cliente y frameworks

Lenguaje	Naturaleza	Responsabilidad
HTML	Marcado por etiquetas, no es lenguaje de programación	Semántica y estructura
CSS	Declarativo de diseño	Aspecto, maquetación, responsive
JavaScript	Dinámico, débilmente tipado, orientado a eventos	Lógica, eventos, validación, DOM

Fechas que se preguntan:

- **1995.** Brendan Eich crea JavaScript en diez días para Netscape. Nació como Mocha y pasó por LiveScript.

- **1996.** Microsoft lanza JScript para Internet Explorer 3.0 por ingeniería inversa. Empieza la guerra de navegadores.
- **1997.** Netscape entrega la especificación a Ecma International: nace ECMAScript, la norma ECMA-262. JavaScript es una implementación.
- **1999.** ES3 consolida el lenguaje: expresiones regulares y `try/catch`.
- **ES4.** Se propuso y se abandonó por complejidad y falta de consenso. De ahí salió Harmony y luego ES5.
- **2005.** Jesse James Garrett acuña AJAX. `XMLHttpRequest` permite actualizar una página sin recargarla.
- **2006.** jQuery, Prototype y MooTools unifican las diferencias del DOM entre navegadores.
- **2009.** ES5: modo estricto, `forEach`, `map`, `filter`, `reduce`, `JSON`, getters y setters. Google lanza Chrome con V8 y Ryan Dahl crea Node.js sobre ese motor.
- **2015.** ES6 o ECMAScript 2015: `let`, `const`, clases, funciones flecha, `map`, `filter` y `reduce`. Desde entonces el TC39 publica una versión al año.

Framework	Autor	Señas de identidad
ReactJS	Meta	Componentes con estado, DOM virtual, JSX
Angular	Google	TypeScript, RxJS, inyección de dependencias
Vue.js	Evan You	DOM virtual, curva progresiva, se usa con Laravel

El **DOM virtual** es una copia ligera del DOM en memoria: el framework compara, calcula las diferencias mínimas (reconciliación) y actualiza solo los nodos que cambian.

Las tres formas de declarar una función no se comportan igual:

- **Declaración:** `function saludar() {}` tiene hoisting, se puede llamar antes de escribirla.
- **Expresión:** `const duplicar = function () {}` no se puede usar antes de la línea donde se asigna.
- **Flecha:** `const sumar = (a, b) => a + b;` y `const cuadrado = x => x * x;` con retorno implícito si el cuerpo es una línea.
- **Trampa:** JSX no es un lenguaje, es una extensión de JavaScript que se compila a JavaScript.
- **Trampa:** React no sustituye al DOM, lo usa; lo que optimiza es cuándo y cómo se modifica.
- **Trampa:** la expresión de función y la función flecha asignadas a `const` no hoistean: llamarlas antes lanza `ReferenceError: Cannot access 'x' before initialization`.
- **Trampa:** elegir Angular para una práctica pequeña. La curva y el tipado no compensan.

1.4 Scripts y programación tradicional

Un script es una secuencia de comandos que interpreta un intérprete. Las cinco diferencias que se preguntan:

Criterio	Lenguaje tradicional	Script
Compilación	Se compila antes de ejecutar	Se interpreta línea a línea
Ejecución	Standalone o máquina virtual	Dentro de un anfitrión: el navegador
Código	Construye sus estructuras	Reutiliza las del anfitrión: DOM, red
Errores	En compilación: no hay binario	En tiempo de ejecución
Ejemplos	C, C++, Java, Swift	JavaScript, PHP, Python, Shell

PHP es un lenguaje de script aunque se ejecute en el servidor. JavaScript también corre fuera del navegador: en consola con Node.js y empaquetado con Electron o pkg.

	Java	**JavaScript**
Tipo	Tradicional	Script
Tipado	Fuerte	Débil
Ejecución	Bytecode sobre la JVM	Interpretado en el navegador
Paradigma	Orientado a objetos	Orientado a eventos

Los motores modernos aplican JIT, así que "JavaScript no se compila nunca" es falso.

El objeto `Date`. Las fechas no son un tipo primitivo: son instancias de `Date` y representan una instantánea congelada, no un reloj que avanza. Guardan milisegundos desde el 1 de enero de 1970 a las 00:00:00 UTC.

- Los meses van de 0 a 11: `0` es enero y `11` diciembre. Los días van de 1 a 31.
- `new Date(2026)` no es el año 2026: son 2026 milisegundos después de 1970.
- Un año entre 0 y 99 se interpreta como siglo XX: `new Date(95, 5, 15)` es el 15 de junio de 1995.
- Hay desbordamiento automático: `new Date(2026, 15, 20)` es abril de 2027 y `new Date(2026, 5, 35)` es el 5 de julio.
- `getMonth()` devuelve 0–11, `getDate()` el día del mes y `getDay()` el día de la semana con 0 = domingo.
- `getTime()` y `Date.now()` devuelven milisegundos. Para base de datos se usa `toISOString()`, que va en UTC.
- Truco: el día 0 del mes siguiente es el último día del mes que se pide.
- **Trampa:** enseñar `getMonth()` directamente a un usuario que espera "septiembre" y darle un 9 que significa octubre.
- **Trampa:** comparar dos fechas con `===`. Son objetos distintos: hay que comparar `getTime()`.
- **Trampa:** `new Date("2026/02/28")` no es ISO y el análisis puede variar. Siempre `new Date("2026-02-28")`.
- **Trampa:** confundir el timestamp en segundos con el de milisegundos.

1.5 Integración de JavaScript en HTML

`<script>` es el contenedor que define el W3C para insertar o enlazar código ejecutable. En HTML5 el navegador asume JavaScript: `type="text/javascript"` se acepta por compatibilidad, pero ya no hace falta. Nunca se cierra de forma abreviada: `<script src="x.js" />` no funciona porque HTML no es XML, y el resto de la página deja de mostrarse.

	Embebido	**Externo (recomendado)**
Dónde	Entre `<script>` y `</script>`	Archivo `.js` enlazado con `src`
Resultado visual	Idéntico	Idéntico
Caché	No se aprovecha	Se descarga una vez y se reutiliza
Mantenimiento	Bloques dispersos por el HTML	Un archivo para todas las páginas
Cuándo	Pocas líneas de una página fija	Cualquier proyecto real

El `<script>` en el `<head>`. El navegador lee el HTML de arriba abajo y detiene el análisis cuando encuentra un script. Como el `<body>` está más abajo, sus elementos todavía no existen en el DOM.

- Si en el `head` solo se **declara** una función, funciona: declarar no ejecuta. La llamada se hace después, desde el `body`.
- Si en el `head` se **toca el DOM** con `getElementById`, devuelve `null` y salta `TypeError: Cannot read properties of null`.
- Soluciones, de mejor a peor: script al final del `body`, `defer` en el `head` para scripts externos, o escuchar `DOMContentLoaded`. Mover el script al final del `head` no arregla nada, porque el `body` sigue estando después.

	`defer`	`async`
Descarga	En segundo plano	En segundo plano
Ejecución	Al terminar de parsear el HTML	En cuanto termina de descargar
Orden entre scripts	Se respeta	No se respeta
Uso típico	Código que necesita el DOM	Analítica y terceros

- **Trampa:** `defer` y `async` solo funcionan en scripts externos, con `src`. En un `<script>` embebido no hacen nada.
- **Trampa:** `addEventListener('click', cambiar())` ejecuta la función al cargar y el clic no hace nada. Se pasa la referencia, sin paréntesis: `cambiar`. En el atributo HTML es al revés: `onclick="cambiar()"`.
- **Trampa:** un 404 en el panel Red suele ser la ruta: `./js/logica.js`, no `logica.js`.
- Abrir el HTML con doble clic y esperar que funcione `fetch()` no funciona: el protocolo `file://` lo bloquea. Hace falta un servidor local.

1.6 Herramientas de programación y prueba

Editor	**Uso**
Visual Studio Code	Estándar de la industria: TypeScript, ESLint, Prettier, Git integrado
Notepad++	Scripts sueltos y archivos grandes
Vim / Neovim	Terminal, muy ligero
WebStorm	El mejor motor de refactorización
Cursor, Windsurf, Zed	Forks o editores nuevos con IA integrada

VS Code y VSCodium se ven igual. La diferencia es la licencia y la telemetría: VSCodium es el mismo código libre compilado sin rastreo.

Panel de DevTools	**Para qué sirve**
Consola	Ejecutar JavaScript en vivo, ver `console.log()` y los errores en rojo
Fuentes	Poner breakpoints, congelar la ejecución y ver variables y la pila de llamadas
Red	Ver peticiones, códigos 200, 404 y 500, tiempos y tamaños

- **Live Server** levanta un servidor local y recarga el navegador al guardar.
- **Quokka.js** ejecuta el JavaScript dentro del editor y muestra el resultado junto a la línea.
- **Error Lens** convierte la línea ondulada en el mensaje de error completo al final de la línea.

Git es el sistema de control de versiones distribuido; GitHub es la plataforma que aloja repositorios, con pull requests, issues e integración continua.

- **Trampa:** un breakpoint en una línea que nunca se ejecuta no se alcanza jamás.

- **Trampa:** guardar sin guardar. Live Server vigila el disco: si el editor tiene cambios pendientes, recarga la versión anterior.
- Un editor ligero no indexa un proyecto de React ni gestiona dependencias: para eso hace falta un IDE.

Preguntas trampa de la Unidad 1

- **¿Cuántos módulos tiene el navegador?** Siete: interfaz, motor del navegador, motor de renderizado, motor de JavaScript, red, UI backend y almacenamiento.
- **¿El motor de renderizado y el de JavaScript son el mismo?** No. Están separados y se avisan.
- **¿Qué motor usa cada navegador?** Chrome, Edge, Brave y Opera usan Blink y V8; Firefox, Gecko y SpiderMonkey; Safari, WebKit y JavaScriptCore.
- **¿Qué navegador usa Chrome en iPhone?** WebKit, igual que Safari, porque lo impone Apple.
- **¿Los cuatro pasos del renderizado?** DOM y CSSOM, Render Tree, Layout y Paint.
- **¿Qué queda fuera del Render Tree?** El `<head>` y todo lo que tiene `display: none`.
- **¿Qué pasa si un script pesado está en el head?** El navegador detiene el análisis del HTML y la pantalla se queda en blanco mientras se descarga y ejecuta.
- **¿`document` es hermano de `window`?** No, es una propiedad: `window.document`.
- **¿`alert("x")` y `window.alert("x")` son iguales?** Sí, porque lo global cuelga de `window`.
- **¿Cuánto mide una cookie?** Hasta 4 KB, y viaja al servidor en cada petición. `localStorage` guarda entre 5 y 10 MB.
- **¿`innerHTML` o `textContent`?** `innerHTML` interpreta marcado y es vulnerable a XSS; `textContent` inserta texto plano.
- **¿Qué hace `document.write()` tras la carga?** Borra el documento entero y deja solo lo escrito.
- **¿Qué efecto tiene `alert()`?** Bloquea el hilo principal: congela animaciones y eventos.
- **¿Reflow o repaint?** Reflow recalcula geometría y desplaza lo que rodea; repaint solo cambia píxeles.
- **¿Qué es ECMAScript?** La especificación ECMA-262. JavaScript es su implementación.
- **¿Quién creó JavaScript y en cuánto tiempo?** Brendan Eich, en diez días, en 1995, para Netscape.
- **¿Qué fue JScript?** La implementación de Microsoft para Internet Explorer 3.0, hecha por ingeniería inversa en 1996.
- **¿Qué pasó con ES4?** Se abandonó por complejidad y falta de consenso; de ahí salió ES5.
- **¿Quién acuñó AJAX y cuándo?** Jesse James Garrett en 2005, con `XMLHttpRequest`.
- **¿Qué trajo ES5?** Modo estricto, métodos funcionales de array, JSON, getters y setters.
- **¿Qué es Node.js?** El entorno que Ryan Dahl creó sobre V8 en 2009 para llevar JavaScript al servidor.
- **¿Qué es el DOM virtual?** Una copia del DOM en memoria; el framework compara y actualiza solo lo que cambia.
- **¿JSX es un lenguaje?** No, es una extensión de JavaScript.
- **¿JavaScript se compila?** Se interpreta, con compilación JIT de las funciones que más

se ejecutan. Decir que nunca se compila es falso.

- **¿PHP es tradicional o script?** Script, aunque corra en el servidor.
- **¿Qué es la época Unix?** El 1 de enero de 1970 a las 00:00:00 UTC. Un día son 86 400 000 ms.
- **¿Qué devuelve `getDay()`?** El día de la semana, con 0 = domingo.
- **¿Para guardar una fecha en base de datos?** `toISOString()`, que va en UTC.
- **¿`defer` o `async`?** `defer` espera al HTML y respeta el orden; `async` va en cuanto llega y no lo respeta. Ninguno funciona en scripts embebidos.
- **¿Se puede cerrar `<script src="x.js" />`?** No: HTML no es XML y el resto de la página no se muestra.
- **¿Hace falta `type="text/javascript"`?** No, en HTML5 es opcional.
- **¿Qué extensión recarga el navegador al guardar?** Live Server. Quokka ejecuta dentro del editor y Error Lens muestra el error completo.
- **¿Git y GitHub son lo mismo?** No: Git es el control de versiones; GitHub, la plataforma que aloja los repositorios.

Unidad 2 — Código

La Unidad 2 se examina prediciendo salidas. Cada ejemplo trae el código mínimo, lo que imprime y la regla que hay detrás. Si solo hay tiempo para una cosa, que sea leer las viñetas **Trampa**: son los errores que se repiten.

2.1 Identificadores, ASI y ECMAScript

Un identificador puede empezar por letra, `$` o `_`, y después admite dígitos. No puede empezar por número ni ser una palabra reservada. JavaScript distingue mayúsculas de minúsculas en todo: identificadores, palabras reservadas y funciones nativas. El juego de caracteres es Unicode, así que los acentos son válidos dentro de un nombre.

Código	Error que lanza
`let 1erUsuario = "Ana";`	`SyntaxError: Invalid or unexpected token`
`let class = "2 DAW";`	`SyntaxError: Unexpected token 'class'`
`let precio-final = 10;`	`SyntaxError: Unexpected token '-'`
`let nombre usuario = "Ana";`	`SyntaxError: Unexpected identifier 'usuario'`

Los cuatro son errores de análisis: el script no llega a ejecutar ni la primera línea. El guion se lee como resta y el espacio separa dos palabras, así que el motor no ve un nombre, ve una expresión rota.

Las convenciones no son sintaxis, pero se preguntan: `camelCase` para variables y funciones (`totalFactura`), `PascalCase` para clases y componentes (`FacturaService`), `UPPER_SNAKE_CASE` para constantes de diseño (`MAX_REINTENTOS`), el prefijo `$` para nodos del DOM y `_` para lo que se quiere marcar como privado.

Ejemplo 1 — El `return` y la llave en línea distinta

```
function obtenerUsuario() {
  return
  {
    nombre: "Ana"
  };
}
console.log(obtenerUsuario());

undefined
```

- El ASI ve el salto de línea detrás de `return`, que no puede continuar la expresión, y escribe el punto y coma por su cuenta. La función devuelve `undefined`.
- El objeto de las líneas siguientes queda como un bloque suelto que nadie lee: no da error porque un bloque con una etiqueta `nombre:` y una cadena es sintaxis válida.
- **Trampa:** la llave de apertura va en la misma línea que el `return`: `return { nombre: "Ana" };`. Es el fallo de estilo que más puntos cuesta.

Ejemplo 2 — La misma trampa con dos propiedades

```
function crearConfiguracion() {
  return
  { modo: "oscuro", puerto: 8080 };
}
console.log(crearConfiguracion());

SyntaxError: Unexpected token ':'
```

- Con una propiedad el bloque colaba; con dos y una coma, el motor interpreta `{ modo: "oscuro"` como bloque y la coma no puede continuar la sentencia. El error apunta al `:` de `puerto`.
- El mensaje no menciona el `return` ni el ASI. Si ves un `SyntaxError` en una línea que "está bien", mira la anterior.
- **Trampa:** un `SyntaxError` de este tipo no se captura con `try/catch`: el archivo no se ejecuta.

Ejemplo 3 — Mayúsculas y minúsculas

```
let totalVentas = 500;
let TotalVentas = 1200;
console.log(totalVentas);
console.log(TotalVentas);

500
1200
```

- Son dos variables distintas y conviven sin conflicto. No hay aviso: si te equivocas de caja, trabajas con otra variable.
- Lo mismo pasa con `alert` y `Alert`: la segunda lanza `ReferenceError: Alert is not defined`. No es un error de sintaxis, es una referencia que no existe.
- **Trampa:** `console.log(Totalventas)` no da error de compilación; da `ReferenceError` en tiempo de ejecución y para el script.

Ejemplo 4 — `typeof` de una variable que no existe

```
console.log(typeof variableNoDeclarada);
console.log(typeof window);

undefined
undefined
```

- `typeof` es la única operación que no lanza con un nombre no declarado: devuelve la cadena `"undefined"`.
- La segunda línea da `undefined` porque en Node no existe `window`. En el navegador devuelve `"object"`, porque `window` es el objeto global.
- **Trampa:** con una variable en zona muerta temporal, `typeof` sí lanza `ReferenceError`. No es un escudo universal (ver 2.3).

ECMAScript en cuatro líneas

- **ECMAScript** es la norma ECMA-262; JavaScript es la implementación que hacen V8, SpiderMonkey y JavaScriptCore.
- **ES6 o ES2015** es el corte importante: `let`, `const`, clases, flechas y métodos de array. Desde 2015 se publica una versión al año y las propuestas pasan por cuatro fases del TC39.
- **TypeScript y JSX no son lenguajes del navegador**: se transpilan a JavaScript antes de servirse.
- El **punto y coma** no es obligatorio por el ASI, pero se escribe siempre: el ASI adivina y a veces adivina mal.

2.2 Variables, tipos y operadores

	`var`	`let`	`const`
Ámbito	Función	Bloque	Bloque
Se puede redeclarar	Sí	No	No
Se puede reasignar	Sí	Sí	No
Sin valor inicial	`undefined`	`undefined`	`SyntaxError`
Antes de su línea	`undefined`	TDZ	TDZ

Ejemplo 1 — `var` redeclara, reasigna y cambia de tipo

```
var nombre = "Juan";
var nombre;
console.log(nombre);

var x = 5, y = 6, z = x + y;
console.log(z);

var dato;
console.log(dato);
dato = 5;
console.log(typeof dato);
dato = "John";
console.log(typeof dato, dato);
```

```
Juan
11
undefined
number
string John
```

- `var nombre;` sin valor no borra nada: la redeclaración es legal y conserva el contenido anterior.
- `dato` existe desde el principio con valor `undefined`; después cambia de número a cadena sin protestar. Eso es el tipado dinámico.
- **Trampa:** `typeof dato` devuelve `"number"` o `"string"` según lo que contenga en ese momento, no según cómo se declaró.

Ejemplo 2 — `let` no admite dos declaraciones

```
let contador = 0;
let contador = 1;
```

```
SyntaxError: Identifier 'contador' has already been declared
```

- El error es de análisis, no de ejecución: no se imprime nada, ni siquiera el `0` de la primera línea.
- Vale igual para `let` y `const`, y también para mezclar `let` con `var` en el mismo ámbito.
- **Trampa:** en la consola del navegador, repetir un `let` da el mismo error y no hay forma de deshacerlo sin recargar la página.

Ejemplo 3 — `const` exige valor y no se reasigna

```
const PI;
```

```
SyntaxError: Missing initializer in const declaration
```

```
const IVA = 0.21;
IVA = 0.10;
```

```
TypeError: Assignment to constant variable.
```

- El primero es de sintaxis: `const` necesita inicializador en la misma declaración.
- El segundo es de ejecución: la constante existe, pero el nombre está atado al valor. `TypeError`, no `SyntaxError`.
- **Trampa:** `const` no congela el contenido, solo impide reasignar el nombre (siguiente ejemplo).

Ejemplo 4 — Una constante con contenido mutable

```
const usuario = { nombre: "Marcos", edad: 20 };
usuario.nombre = "Laura";
console.log(usuario.nombre);

const notas = [5, 8];
notas.push(7);
console.log(notas.length, notas[2]);
```

```
Laura
3 7
```

- `const` protege la referencia, no el objeto: se pueden cambiar propiedades y añadir

elementos al array.

- `usuario = { nombre: "Pedro" }` sí lanzaría `TypeError: Assignment to constant variable.`
- **Trampa:** para congelar de verdad hay que usar `Object.freeze(usuario)`, y aun así es superficial: no baja a los objetos anidados.

La tabla de `typeof`

Expresión	Devuelve	Nota
`typeof 42`	`"number"`	También `NaN` e `Infinity`
`typeof "Hola"`	`"string"`	
`typeof true`	`"boolean"`	
`typeof undefined`	`"undefined"`	
`typeof 100n`	`"bigint"`	El sufijo `n`
`typeof Symbol("id")`	`"symbol"`	
`typeof { a: 1 }`	`"object"`	
`typeof [1, 2, 3]`	`"object"`	Un array no tiene tipo propio
`typeof function () {}`	`"function"`	El único caso especial
`typeof NaN`	`"number"`	`NaN` es un número inválido
`typeof null`	`"object"`	Anomalía histórica de 1995

Para distinguir un array de un objeto se usa `Array.isArray(valor)`, que devuelve `true` o `false`. Para saber si algo es `null` se compara con `=== null`.

Ejemplo 5 — Aritmética y precisión

```
console.log(10 / 4);
console.log(10 % 3);
console.log(2 ** 3);
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log(10 / 0);
console.log(-10 / 0);
console.log("hola" * 2);
console.log(-7 % 2);
console.log(Number.MAX_SAFE_INTEGER + 2);
```

```
2.5
1
8
0.30000000000000004
false
Infinity
-Infinity
NaN
-1
9007199254740992
```

- `/` siempre devuelve decimal: no hay división entera, para eso está `Math.floor`.
- `%` conserva el signo del dividendo, por eso `-7 % 2` es `-1` y no `1`.
- `0.1 + 0.2` es el ejemplo canónico del coma flotante IEEE 754. No se compara con `===`: se comparan céntimos enteros o se usa una tolerancia.
- Dividir entre cero no lanza excepción: da `Infinity`. `0 / 0` daría `NaN`.
- `"hola" * 2` da `NaN` porque el operador aritmético convierte a número y no puede.
- **Trampa:** por encima de `Number.MAX_SAFE_INTEGER` se pierde precisión en silencio (`...991 + 2` da `...992`). Para eso está `BigInt`, con sufijo `n`.

Ejemplo 6 — Postincremento y preincremento

```
let i = 2;
console.log(i++ * ++i);
console.log(i);

let a = 5;
let b = a++;
console.log(b, a);

let c = 5;
let d = ++c;
console.log(d, c);
```

```
8
4
5 6
6 6
```

- `i++` devuelve el valor viejo (2) y después suma; `++i` suma primero y devuelve el nuevo (4). El producto es `2 * 4 = 8` y `i` acaba en 4.
- `b = a++` deja `b` con 5 y `a` con 6: el valor que se asigna es el de antes del incremento.
- `d = ++c` deja los dos en 6: primero incrementa, luego asigna.
- **Trampa:** el `i` final es 4, no 5: solo hay un `++i` y un `i++`, es decir, dos incrementos sobre el 2.

Ejemplo 7 — Precedencia con unarios: el ejercicio del 2.5

```
let x = 3;
let y = 4;
const resultado = x++ * ++y + (x & 1);
console.log(resultado);
console.log(x);
console.log(y);
```

```
15
4
5
```

- Paso a paso: `x++` aporta 3 y deja `x` en 4; `++y` aporta 5 y deja `y` en 5; el producto es 15.
- El paréntesis se evalúa después del postincremento, así que `x & 1` es `4 & 1 = 0`. Total: `15 + 0 = 15`.
- **Trampa:** calcular `x & 1` con el 3 inicial da 1 y un total de 16. El paréntesis no congela el valor de `x`: solo ordena la operación.

Ejemplo 8 — Precedencia y asociatividad

```
console.log(2 + 3 * 4);
console.log((2 + 3) * 4);
console.log(1 + 2 + "3");
console.log("1" + 2 + 3);
console.log(typeof 1 + 2);
console.log(2 ** 3 ** 2);
console.log((1, 2, 3));
console.log(1, 2, 3);
```

```
14
20
33
123
number2
512
3
1 2 3
```

- `*` va antes que `+`; los paréntesis mandan. En `1 + 2 + "3"` primero se suma 3 y luego se concatena: la cadena `"33"`.
- `+` es asociativo por la izquierda: en cuanto aparece una cadena, todo lo que sigue se concatena. `"1" + 2 + 3` es `"123"`.
- `typeof` tiene más precedencia que `+`, así que `typeof 1 + 2` es `"number" + 2`, la cadena `"number2"`.
- **`` es asociativo por la derecha: `2 3 2` es `2 9 = 512, no 8 ** 2 = 64`.**
- El operador coma devuelve el último valor, `3`; dentro de `console.log` la coma separa argumentos y por eso imprime tres.
- **Trampa:** `1 + 2 + "3"` y `"1" + 2 + 3` no dan lo mismo. El orden de evaluación decide.

Ejemplo 9 — `&&` y `||` devuelven un operando, no un booleano

```
console.log("Usuario" && 0 && "Admin");
console.log("Servidor OK" && 200);
console.log(false || "Por defecto");
console.log("" || 0 || null);
console.log(!!"texto");
console.log(!!0);
console.log(![]);
console.log(!!"false");
```

```
0
200
Por defecto
null
true
false
true
true
```

- `&&` devuelve el primer valor falsy que encuentra o el último si todos son truthy. `||` devuelve el primer truthy o el último si todos son falsy.
- `!!` es la forma corta de `Boolean()`: convierte a booleano de verdad.
- `[]` y `{}` son truthy aunque estén vacíos. `"false"` es una cadena no vacía: también truthy.
- **Trampa:** `"" || 0 || null` da `null`, no `false`: el operador devuelve el operando, no el resultado de la comparación.

Ejemplo 10 — `||` frente a `??` con un descuento de 0

```
function fijarDescuento(descuento) {
  const tasaA = descuento || 15;
  const tasaB = descuento ?? 15;
  console.log(`Tasa A: ${tasaA} | Tasa B: ${tasaB}`);
}
fijarDescuento(0);
fijarDescuento(undefined);
fijarDescuento(25);

Tasa A: 15 | Tasa B: 0
Tasa A: 15 | Tasa B: 15
Tasa A: 25 | Tasa B: 25
```

- `||` considera ausencia cualquier valor falsy: `0`, `""`, `false`, `NaN`. Con un descuento de 0 aplica 15, que es un error de negocio.
- `??` solo sustituye `null` y `undefined`, que es lo que significa "no me han pasado nada".
- **Trampa:** mezclar `??` con `||` o `&&` sin paréntesis es un `SyntaxError: missing ) after argument list`. Se escribe `(a ?? b) || c`.

Ejemplo 11 — Asignación lógica

```
let usuario = null;
usuario ??= "Invitado";
console.log(usuario);

let puerto = 0;
puerto ??= 8080;
console.log(puerto);

let titulo = "";
titulo ||= "Sin título";
console.log(titulo);

let nombre = "Ana";
nombre &&= nombre.toUpperCase();
console.log(nombre);

Invitado
0
Sin título
ANA
```

- `??=` solo asigna si el valor es `null` o `undefined`: el puerto 0 se respeta.
- `||=` asigna con cualquier falsy, por eso la cadena vacía se sustituye.
- `&&=` asigna solo si el valor es truthy; aquí sirve para transformar el nombre.
- **Trampa:** `0` es un puerto válido y `""` puede ser un título válido. Elegir `||=` donde tocaba `??=` cambia el comportamiento sin dar error.

Ejemplo 12 — `in`, `delete` e `instanceof`

```
const alumno = { id: 1001, curso: "2 DAW", activo: true };
console.log("curso" in alumno);
console.log("email" in alumno);
delete alumno.activo;
console.log("activo" in alumno);
console.log(alumno instanceof Object);
console.log([1, 2] instanceof Array);
const hoy = new Date();
console.log(hoy instanceof Date);
```

```
true
false
false
true
true
true
```

- `in` comprueba si la propiedad existe, incluidas las heredadas del prototipo. No mira el valor: una propiedad con `undefined` sigue estando.
- `delete` borra propiedades de objetos; no borra variables declaradas ni funciones.
- `instanceof` recorre la cadena de prototipos. Un array es `Array` y también `Object`.
- **Trampa:** `delete` sobre un array deja un hueco, no reordena: `delete a[1]` mantiene `length` 3 y deja `a[1]` en `undefined`.

Operadores por grupos

Grupo	Operadores
Aritméticos	`+ - * / % **`
Comparación	`== != === !== < <= > >=`
Lógicos	`&& || ?? !`
Asignación	`= += -= *= /= %= **= ??= ||= &&=`
Unarios	`typeof ++ -- + - ! void delete`
Especiales	`in instanceof ?. ...`
Bit a bit	`& | ^ << >> >>>`

`?.` es el encadenamiento opcional: `usuario?.direccion?.calle` devuelve `undefined` en vez de lanzar si falta un eslabón.

Resultados que caen seguro

Expresión	Resultado
`"5" + 2`	`"52"`
`"5" - 2`	`3`
`true + true`	`2`
`[] + {}`	`"[object Object]"`
`[] + []`	`""`, cadena vacía
`[1, 2] + [3]`	`"1,23"`
`null + 1`	`1`
`undefined + 1`	`NaN`
`"10" < "9"`	`true`, compara texto
`"10" < 9`	`false`, compara números
`3 > 2 > 1`	`false`, porque `true > 1` es `1 > 1`
`parseInt("08")`	`8`, la base 10 es la de por defecto

2.3 Ámbito, hoisting y closures

Antes de ejecutar una línea, el motor reserva los nombres de las declaraciones del ámbito. Eso es el hoisting: `var` se queda en `undefined`, `let` y `const` quedan en zona muerta temporal (TDZ) hasta su línea, y las funciones declaradas se elevan completas.

Ejemplo 1 — Hoisting de `var`

```
console.log(precioArticulo);
var precioArticulo = 50;
console.log(precioArticulo);

undefined
50
```

- La declaración se eleva, la asignación no. En la primera línea el nombre existe y vale `undefined`.
- No lanza `ReferenceError` justamente porque la declaración ya está registrada.
- **Trampa:** "elevado" no significa "inicializado". Si operas con ese `undefined` (`precioArticulo * 2`) obtienes `NaN` sin ningún error.

Ejemplo 2 — La zona muerta temporal de `let`

```
console.log(b);
let b = 10;
console.log(b);

ReferenceError: Cannot access 'b' before initialization
```

- `let` también se eleva, pero queda en TDZ: existe y no se puede tocar hasta su línea.
- El mensaje es distinto de `b is not defined`, que es el de una variable que no se ha declarado en ningún sitio. En el examen, esa diferencia es la respuesta.
- **Trampa:** la TDZ empieza al abrir el bloque, no en la línea del `let`. Cualquier acceso anterior, aunque esté en otra función, falla.

Ejemplo 3 — `typeof` no salva de la TDZ

```
console.log(typeof c);
let c = 10;

ReferenceError: Cannot access 'c' before initialization
```

- Con una variable no declarada, `typeof` devuelve `"undefined"`; con una declarada pero en TDZ, lanza.
- Es la pareja de trampas de examen: `typeof` no lanza por la izquierda, pero sí por la derecha.
- **Trampa:** responder "devuelve undefined" aquí es el error más habitual de todo el tema.

Ejemplo 4 — Sombra dentro de una función

```
let usuario = "Carlos";

function configurarPerfil() {
  console.log("El usuario actual es:", usuario);
  let usuario = "Marta";
}
configurarPerfil();
```

```
ReferenceError: Cannot access 'usuario' before initialization
```

- El `let usuario` de dentro crea una variable local que eclipsa la global en **toda** la función, también en la línea de antes.
- El `console.log` ya está en TDZ de la variable local, así que no llega a leer la global.
- **Trampa:** el error no está en el `let`, está en la línea anterior. Y si en vez de `let` fuera `var`, el log imprimiría `undefined` sin error.

Ejemplo 5 — Sombra deliberada de una constante

```
const TASA = 0.21;

function calcularTarifa(precio) {
  const TASA = 0.10;
  return precio * TASA;
}

console.log(TASA);
console.log(calcularTarifa(200));
console.log(TASA);
```

```
0.21
20
0.21
```

- La constante local vive dentro de la función y no toca la global. Al salir, vuelve a verse la de fuera.
- Es útil para aislar un cálculo, y peligroso si el nombre se reutiliza sin querer: el resultado cambia y no hay ningún error.
- **Trampa:** si hubiera un `var TASA` en la función, la global seguiría intacta, pero la local se elevaría a `undefined` y el cálculo daría `NaN`.

Ejemplo 6 — Closure: una función que recuerda

```
function crearContador() {
  let cuenta = 0;
  return function () {
    cuenta += 1;
    return cuenta;
  };
}

const siguiente = crearContador();
console.log(siguiente());
console.log(siguiente());
console.log(siguiente());
console.log(crearContador()());
```

```
1
2
3
1
```

- La función interna conserva el ámbito donde nació: `cuenta` sobrevive a `crearContador()` y se comparte entre llamadas.
- Cada llamada a `crearContador()` crea un ámbito nuevo, por eso el último `1` empieza de cero.
- **Trampa:** `console.log(siguiente)` sin paréntesis imprime la función, no el número. Y `crearContador()()` es llamar a la devuelta en el acto.

Ejemplo 7 — `var` no respeta el bloque; `let` sí

```
{
  var fuga = "visible fuera";
  let encerrada = "solo aquí";
  console.log(encerrada);
}
console.log(fuga);
console.log(typeof encerrada);
```

```
solo aquí
visible fuera
undefined
```

- `var` se declara en la función o en el global, así que el bloque no la encierra.
- `let` muere al cerrar la llave: fuera no existe y `typeof` devuelve `"undefined"` sin lanzar, porque no está declarada en ese ámbito.
- **Trampa:** ese `undefined` no es TDZ, es ausencia de declaración. Si en vez de `typeof` escribes `console.log(encerrada)`, salta `ReferenceError: encerrada is not defined`.

Ejemplo 8 — El bucle con temporizadores: `var` frente a `let`

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var:", i), 100);
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let:", j), 100);
}
console.log("Fin de los bucles (síncrono)");
```

```
Fin de los bucles (síncrono)
var: 3
var: 3
var: 3
let: 0
let: 1
let: 2
```

- Los temporizadores no se ejecutan hasta que la pila se vacía, por eso el mensaje síncrono sale primero aunque esté escrito al final.
- Con `var` hay una sola variable `i` compartida por las tres funciones; cuando se ejecutan, el bucle ya terminó y vale 3.
- Con `let` cada iteración crea un enlace nuevo con su valor, y cada función captura el suyo.

- **Trampa:** el orden de las líneas del código no es el orden de la salida. Primero todo lo síncrono, después las tareas programadas.

Ejemplo 9 — `"use strict"` y la global accidental

```
"use strict";

function procesarCalculo() {
  resultado = 100;
  return resultado;
}
procesarCalculo();

ReferenceError: resultado is not defined
```

- Sin modo estricto, asignar a un nombre no declarado crea una propiedad global: `resultadoFuga = 100` deja `globalThis.resultadoFuga` en 100.
- Con `"use strict"` esa misma línea lanza `ReferenceError`. Es la primera diferencia práctica del modo estricto.
- Los módulos ES y las clases son estrictos por defecto, aunque no se escriba la directiva.
- **Trampa:** el modo estricto no convierte las variables en constantes ni impide reasignar un `let`.

Ejemplo 10 — Funciones: declaración frente a expresión

```
console.log(duplicar(4));

function duplicar(n) {
  return n * 2;
}

console.log(triplicar(4));

const triplicar = function (n) {
  return n * 3;
};

8
ReferenceError: Cannot access 'triplicar' before initialization
```

- La declaración se eleva completa: se puede llamar antes de su línea y funciona.
- La expresión asignada a `const` no: está en TDZ y la llamada anticipada lanza `ReferenceError`.
- **Trampa:** con `var triplicar = function () {}` no hay `ReferenceError`, hay `TypeError: triplicar is not a function`, porque el nombre existe con valor `undefined`.

Resumen de ámbito y hoisting

Declaración	Se eleva	Antes de su línea	Ámbito
`function f() {}`	Completa	Se puede llamar	Bloque en estricto, función fuera
`var x`	Sí	`undefined`	Función
`let x`	Sí	TDZ, `ReferenceError`	Bloque

Declaración	Se eleva	Antes de su línea	Ámbito
`const x`	Sí	TDZ, `ReferenceError`	Bloque
`class C {}`	Sí	TDZ, `ReferenceError`	Bloque

2.4 Conversiones, truthy/falsy y comparaciones

JavaScript convierte tipos sin pedir permiso. La conversión puede ser explícita (`Number(x)`, `String(x)`, `Boolean(x)`) o implícita, que es la que hace un operador cuando los operandos no encajan: `"5" * 2`, `1 + "2"`, `if (valor)`.

Las conversiones a número

Expresión	Resultado	Por qué
`Number("42")`	`42`	Cadena numérica completa
`Number("42px")`	`NaN`	`Number` exige la cadena entera
`Number("")`	`0`	La cadena vacía vale cero
`Number(" ")`	`0`	Solo espacios, también cero
`Number("0")`	`0`	
`Number(true)`	`1`	
`Number(false)`	`0`	
`Number(null)`	`0`	
`Number(undefined)`	`NaN`	No hay valor que convertir
`Number([])`	`0`	El array vacío se convierte en `""`
`Number([25])`	`25`	Un solo elemento, se usa su texto
`Number([1, 2])`	`NaN`	`"1,2"` no es un número
`Number({})`	`NaN`	`"[object Object]"`

```
console.log(parseInt("42px", 10));
console.log(parseFloat("19.95 euros"));
console.log(parseInt("ancho: 100"));
console.log(+"150");
```

```
42
19.95
NaN
150
```

- `parseInt` lee desde el principio y se para en el primer carácter que no es un dígito; `Number` exige que toda la cadena sea numérica.
- `parseInt("ancho: 100")` es `NaN` porque no empieza por número: no busca dígitos en medio de la cadena.
- El `+` delante es la forma abreviada de `Number`, y `parseFloat` acepta decimales y notación científica.
- **Trampa:** `parseInt` sin base puede interpretar un cero inicial de forma inesperada en motores antiguos. Se escribe siempre `parseInt(texto, 10)`.

Ejemplo 1 — A cadena

```
console.log(String(123));
console.log(String(true));
console.log(String(null));
console.log(String(undefined));
console.log(String([1, 2]));
console.log(String({}));
```

```
123
true
null
undefined
1,2
[object Object]
```

- `String(null)` devuelve la cadena `"null"`, que es truthy y no tiene nada que ver con `null`.
- Un array se convierte con `join(",")`; un objeto, con su `toString()`, que por defecto da `[object Object]`.
- **Trampa:** `"Total: " + objeto` imprime `Total: [object Object]`. Para ver el contenido hay que usar `JSON.stringify(objeto)`.

Ejemplo 2 — La suma concatena y la resta convierte

```
console.log("5" + 2);
console.log("5" - 2);
console.log(true + true);
console.log(true + "1");
console.log([] + {});
console.log([1, 2] + [3]);
console.log([] + 1);
console.log(null + 1);
console.log(undefined + 1);
console.log("12" + "5");
```

```
52
3
2
true1
[object Object]
1,23
1
1
NaN
125
```

- `+` es el único aritmético que concatena: si cualquiera de los dos operandos es cadena, el otro se convierte a cadena.
- `-`, `*`, `/` y `%` siempre convierten a número, por eso `"5" - 2` es 3 y no un error.
- `null` se convierte en 0 y `undefined` en `NaN`, que contamina cualquier operación.
- `"12" + "5"` es `"125"`, no 17: los dos son cadenas y se concatenan.
- **Trampa:** `[] + []` da la cadena vacía, que en la consola se ve como una línea en blanco. Y `[] + {}` da `"[object Object]"`.

Los ocho valores falsy

Valor	`Boolean(valor)`	Nota
`false`	`false`	
`0`	`false`	
`-0`	`false`	
`0n`	`false`	BigInt cero
`""`	`false`	Cadena vacía
`null`	`false`	
`undefined`	`false`	
`NaN`	`false`	
`"0"`	`true`	Cadena no vacía
`" "`	`true`	Un espacio ya es contenido
`[]`	`true`	Array vacío, pero objeto
`{}`	`true`	Objeto vacío, pero objeto
`-1`	`true`	Cualquier número distinto de 0

Son ocho y no hay más. Todo lo demás es truthy, incluidos `"false"`, `"0"`, `[]` y `{}`.

- **Trampa:** `if (arr)` no comprueba si el array está vacío: un array vacío entra en el `if`. Se comprueba con `arr.length > 0`.
- **Trampa:** `if (respuesta)` con `respuesta = "0"` entra, y con `respuesta = 0` no. Si el 0 es válido, hay que comparar con `=== undefined` o usar `??`.

La tabla de la igualdad abstracta

Expresión	Resultado	Regla aplicada
`null == undefined`	`true`	Regla especial: solo son iguales entre sí
`null === undefined`	`false`	Tipos distintos
`null == 0`	`false`	La regla especial no incluye el 0
`undefined == 0`	`false`	Tampoco
`0 == ""`	`true`	`""` se convierte en 0
`0 == "0"`	`true`	`"0"` se convierte en 0
`"" == false`	`true`	Los dos a 0
`"0" == false`	`true`	`"0"` a 0, `false` a 0
`[] == false`	`true`	`[]` a `""` a 0
`[] == ![]`	`true`	`![]` es `false`, y `[] == false`
`[] == ""`	`true`	`[]` se convierte en cadena vacía
`[0] == false`	`true`	`[0]` a `"0"` a 0
`"1" == true`	`true`	`true` a 1
`NaN == NaN`	`false`	`NaN` no es igual a nada

- Las comparaciones relacionales convierten a número: `null <= 0` es `true` y `null >= 0` también, aunque `null == 0` sea `false`.
- Dos cadenas se comparan carácter a carácter por código: `"10" < "9"` es `true` porque `"1"` va antes que `"9"`.
- **Trampa:** por eso `==` no se usa nunca en código real. Se usa `===`, y con `null` se compara explícitamente.

Ejemplo 3 — `NaN`, `isNaN` y `Object.is`

```
console.log(NaN == NaN);
console.log(NaN === NaN);
console.log(Object.is(NaN, NaN));
console.log(0 === -0);
console.log(Object.is(0, -0));
```

```
false
false
true
true
false
```

```
console.log(isNaN("hola"));
console.log(Number.isNaN("hola"));
console.log(isNaN("42"));
console.log(Number.isNaN(NaN));
console.log("hola" === NaN);
```

```
true
false
false
true
false
```

- `NaN` no es igual a nada, ni siquiera a sí mismo. Para preguntar si algo es `NaN` se usa `Number.isNaN`.
- `isNaN` convierte primero el argumento: `isNaN("hola")` es `true` porque `Number("hola")` es `NaN`. `Number.isNaN` no convierte.
- `Object.is` es como `===` salvo en dos casos: `NaN` es igual a `NaN` y `0` no es igual a `-0`.
- **Trampa:** `x === NaN` nunca es `true`. Y `Number.isNaN("hola")` es `false`: una cadena no es un `NaN`, es una cadena.

Ejemplo 4 — `Symbol` y `BigInt`

```
const id = Symbol("uid");
console.log(String(id));
console.log(id === Symbol("uid"));
console.log(Boolean(id));
console.log(500n + " bytes");
console.log(Boolean(0n), Boolean(15n));
```

```
Symbol(uid)
false
true
500 bytes
false true
```

```
const id = Symbol("uid");
console.log("Usuario: " + id);
```

```
TypeError: Cannot convert a Symbol value to a string
```

```
const clave = 9007199254740995n;
console.log(clave + 5);
```

```
TypeError: Cannot mix BigInt and other types, use explicit conversi
ons
```

- Cada `Symbol` es único aunque las descripciones coincidan: `id === Symbol("uid")` es `false`.
- Un `Symbol` no se puede concatenar ni interpolar en una plantilla. Se convierte con `String(id)`.
- `BigInt` no se mezcla con `number`: se opera con `5n` o se convierte con `Number()`, que puede perder precisión.
- **Trampa:** `500n + " bytes"` funciona porque la suma con cadena concatena; `500n + 5` lanza `TypeError`. La concatenación y la aritmética no son lo mismo.

2.5 Condicionales

Ejemplo 1 — Cadena de `if` con retorno temprano

```
function clasificar(nota) {
  if (nota < 0 || nota > 10) return "Fuera de rango";
  if (nota < 5) return "Suspenso";
  if (nota < 7) return "Aprobado";
  if (nota < 9) return "Notable";
  return "Sobresaliente";
}
console.log(
  clasificar(4.9), clasificar(5), clasificar(7),
  clasificar(9), clasificar(11)
);
```

```
Suspenso Aprobado Notable Sobresaliente Fuera de rango
```

- Las condiciones se comprueban de arriba abajo y el primer `return` termina la función. No hacen falta `else`.
- 4.9 cae en `< 5`, 5 ya no, pero sí en `< 7`, y así hasta el final. El último valor no necesita condición.
- **Trampa:** si se cambia el orden y `nota < 7` va antes que `nota < 5`, un 4.9 cae en el primero y se etiqueta como Aprobado.

Ejemplo 2 — `switch` compara con `===`

```
let nivelAcceso = "1";
switch (nivelAcceso) {
  case 1:
    console.log("Nivel administrador");
    break;
  default:
    console.log("No coincide: '1' !== 1");
}
```

```
No coincide: '1' !== 1
```

- El `switch` usa igualdad estricta: la cadena `"1"` no coincide con el número `1`.
- Si ningún `case` coincide, entra el `default`, esté donde esté.
- **Trampa:** aquí no hay conversión de tipos. O se declara `case "1":` o se convierte antes con `Number(nivelAcceso)`.

Ejemplo 3 — Fall-through: sin `break` no se para

```
let opcion = 2;
switch (opcion) {
  case 1:
    console.log("Uno");
  case 2:
    console.log("Dos");
  case 3:
    console.log("Tres");
  default:
    console.log("Default");
}
```

```
Dos
Tres
Default
```

- Al coincidir el `case 2` empieza a ejecutar ahí y sigue hacia abajo, porque ningún `break` corta.
- El `default` del final también se ejecuta, aunque no sea una coincidencia.
- **Trampa:** el fall-through se usa a propósito para agrupar `case`, pero olvidar un `break` es un bug silencioso: imprime de más y no da error.

Ejemplo 4 — `default` en medio

```
let valor = 99;
switch (valor) {
  default:
    console.log("No reconocido");
  case 1:
    console.log("Uno");
  case 2:
    console.log("Dos");
}
```

```
No reconocido
Uno
Dos
```

- El `default` no tiene que ir al final: se ejecuta cuando nada coincide y después sigue cayendo a los `case` siguientes.
- **Trampa:** "el default siempre va al final" es falso, y si está en medio sin `break` arrastra todo lo de abajo.

Ejemplo 5 — `case` agrupados

```
function estacion(mes) {
  switch (mes) {
    case "DICIEMBRE":
    case "ENERO":
    case "FEBRERO":
      return "Invierno";
    case "MARZO":
    case "ABRIL":
    case "MAYO":
      return "Primavera";
    default:
      return "Otra estación";
```

```
  }
}
console.log(estacion("ABRIL"));
console.log(estacion("ENERO"));
console.log(estacion("AGOSTO"));
```

```
Primavera
Invierno
Otra estación
```

- Varios `case` sin código comparten el bloque del siguiente: es la forma limpia de agrupar valores.
- Aquí no hace falta `break` porque cada rama termina con `return`.
- **Trampa:** si un `case` del grupo tuviera una sentencia, solo se ejecutaría para ese valor y luego caería al resto. El grupo funciona porque están vacíos.

Ejemplo 6 — Ternario

```
const nota = 7.5;
const etiqueta =
  nota < 5 ? "Suspenso" :
  nota < 7 ? "Aprobado" :
  nota < 9 ? "Notable" : "Sobresaliente";
console.log(etiqueta);
console.log(`El alumno está ${nota >= 5 ? "aprobado" : "suspenso"}.
`);
```

```
Notable
El alumno está aprobado.
```

- El ternario es una expresión: devuelve un valor y por eso se puede asignar o interpolar dentro de una plantilla.
- Encadenado y partido en líneas se lee aceptablemente; todo en una línea, no.
- **Trampa:** para tres o más ramas conviene `if` o `switch`. Un ternario anidado sin formato es el sitio donde más erratas se cuelan.

Ejemplo 7 — `&&` y `||` como decisión

```
const usuarioLogueado = true;
const abrirPanel = () => console.log("Panel abierto");
usuarioLogueado && abrirPanel();
```

```
Panel abierto
```

```
const config = { tema: "" };
const guardar = () => console.log("Guardado");
config.tema || guardar();
```

```
Guardado
```

- `condicion && accion()` ejecuta la acción solo si la condición es truthy. Si no, el resultado es el valor falsy y no pasa nada.
- `valor || accion()` ejecuta la acción cuando el valor es falsy: sirve como valor por defecto con efecto.
- **Trampa:** si el valor puede ser `0` o `""` legítimos, `||` dispara la acción sin querer. Ahí toca `??` o un `if` explícito.

Ejemplo 8 — Cláusulas de guarda

```
function procesarRegistroUsuario(datos) {
  if (!datos) {
    return { exito: false, error: "Datos no proporcionados" };
  }
  if (datos.edad < 18) {
    return { exito: false, error: "El usuario debe ser mayor de eda
d" };
  }
  if (!datos.email || !datos.email.includes("@")) {
    return { exito: false, error: "Formato de email incorrecto" };
  }
  const usuario = { id: Date.now(), ...datos };
  return { exito: true, usuario };
}
console.log(procesarRegistroUsuario(null).error);
console.log(procesarRegistroUsuario({ edad: 16, email: "a@b.c" }).e
rror);
console.log(procesarRegistroUsuario({ edad: 20, email: "sin-arroba"
 }).error);
console.log(procesarRegistroUsuario({ edad: 20, email: "ana@daw.es"
 }).exito);
```

```
Datos no proporcionados
El usuario debe ser mayor de edad
Formato de email incorrecto
true
```

- Cada guarda valida una precondición y sale. El camino feliz queda al final, con un solo nivel de sangría.
- El orden importa: `datos.edad` se lee después de comprobar que `datos` existe. Al revés, un `null` lanzaría `TypeError`.
- `!datos.email || !datos.email.includes("@")` combina dos comprobaciones: que exista y que tenga arroba.
- **Trampa:** una guarda no es un `else`. Si se escribe `if (!datos) { ... } else { ... }`, se vuelve al anidamiento que se quería evitar.

Ejemplo 9 — `calcularMatricula` con guardas

```
function calcularMatricula(datos) {
  if (!datos || typeof datos !== "object") {
    return { valido: false, error: "Parámetros de matrícula inválid
os." };
  }
  if (typeof datos.precioBase !== "number" || datos.precioBase <= 0
) {
    return { valido: false, error: "El precio base debe ser un núme
ro positivo." };
  }
  if (!["general", "desempleado", "socio"].includes(datos.tipoEstud
iante)) {
    return { valido: false, error: "Tipo de estudiante no reconocid
o." };
  }

  let descuentoPorcentaje = 0;
  if (datos.tipoEstudiante === "desempleado") {
    descuentoPorcentaje = 0.30;
```

```
  } else if (datos.tipoEstudiante === "socio") {
    descuentoPorcentaje = 0.15;
  }

  let total = datos.precioBase * (1 - descuentoPorcentaje);
  if (datos.codigoDescuento === "WEB2026") {
    total = total - 10;
  }
  if (total < 0) {
    total = 0;
  }
  return { valido: true, total: total };
}

console.log(calcularMatricula(null).error);
console.log(calcularMatricula({ precioBase: -20, tipoEstudiante: "g
eneral" }).error);
console.log(calcularMatricula({ precioBase: 100, tipoEstudiante: "e
xtranjero" }).error);
console.log(calcularMatricula({ precioBase: 100, tipoEstudiante: "d
esempleado", codigoDescuento: "WEB2026" }).total);
console.log(calcularMatricula({ precioBase: 100, tipoEstudiante: "s
ocio" }).total);
console.log(calcularMatricula({ precioBase: 5, tipoEstudiante: "gen
eral", codigoDescuento: "WEB2026" }).total);
console.log(calcularMatricula({ precioBase: 100, tipoEstudiante: "g
eneral" }).total);
```

```
Parámetros de matrícula inválidos.
El precio base debe ser un número positivo.
Tipo de estudiante no reconocido.
60
85
0
100
```

- Tres guardas al principio: datos, precio y tipo. Cada una devuelve el error y la función termina ahí.
- El anidamiento baja de cuatro niveles en la versión piramidal a uno: el cálculo vive en el cuerpo principal, sin `else` detrás de ningún `return`.
- `WEB2026` resta 10 después de aplicar el porcentaje: 100 con un 30% son 70, menos 10, 60. Con 5 euros el total sería negativo y se queda en 0.
- **Trampa:** `typeof datos.precioBase !== "number"` rechaza `"100"`, que es una cadena. Si el dato llega de un formulario, hay que convertirlo antes con `Number()`.

Ejemplo 10 — `console.assert`

```
console.assert(1 + 1 === 2, "Esto no se ve: la condición es verdade
ra");
console.assert(1 === 2, "Fallo en Test X: 1 no es 2");
console.log("Fin del script");
```

```
Fin del script

Assertion failed: Fallo en Test X: 1 no es 2
```

- `console.assert` solo escribe cuando la condición es falsa, y lo hace como error. La ejecución sigue: el programa no se detiene.
- La primera línea del ejemplo no imprime nada porque la condición es verdadera.

- **Trampa:** en DevTools el fallo sale en rojo dentro de la consola, no en la página. Y en Node va por la salida de error, así que no aparece si solo se captura `stdout`.

Qué estructura elegir

Situación	Estructura
Dos caminos y una condición simple	`if` / `else`
Varias ramas sobre el mismo valor	`switch`
Elegir entre dos valores y asignarlos	Ternario
Validar precondiciones al principio	Guardas con `return`
Ejecutar algo solo si se cumple	`&&`
Valor por defecto	`??`, o `||` si 0 y `""` no son válidos

2.6 Bucles

Ejemplo 1 — `while`

```
let pase = 0;
while (pase < 3) {
  console.log(`Pase número ${pase + 1}`);
  pase++;
}
console.log("Fin del while");
```

```
Pase número 1
Pase número 2
Pase número 3
Fin del while
```

- La condición se evalúa antes de cada vuelta: si es falsa al principio, el cuerpo no se ejecuta nunca.
- La variable de control se declara fuera y se modifica dentro. Si se olvida el `pase++`, el bucle no termina nunca.
- **Trampa:** un bucle infinito no lanza error: congela la pestaña. En el examen, si no hay incremento, la respuesta es "no termina".

Ejemplo 2 — `do...while`

```
let llaveAcceso = false;

while (llaveAcceso === true) {
  console.log("NUNCA se imprime");
}

do {
  console.log("Se imprime exactamente 1 vez");
} while (llaveAcceso === true);

let contadorPares = 0;
let numeroActual = 1;
do {
  if (numeroActual % 2 === 0) {
    console.log(`Par: ${numeroActual}`);
    contadorPares++;
  }
  numeroActual++;
} while (contadorPares < 5);
```

```
Se imprime exactamente 1 vez
Par: 2
Par: 4
Par: 6
Par: 8
Par: 10
```

- El `while` con la condición falsa no entra ni una vez; el `do...while` ejecuta el cuerpo y comprueba después, así que entra al menos una vez.
- El segundo bucle no sabe cuántas vueltas dará: para cuando ha encontrado cinco pares, en el 10.
- **Trampa:** la condición del `do...while` lleva punto y coma al final. Y ojo con la diferencia: si la condición inicial es falsa, `while` da 0 iteraciones y `do...while` da 1.

Ejemplo 3 — `for`: las tres partes son opcionales

```
const coches = ["Toyota", "Kia", "Citroen", "Ford"];
let i = 2;
for (; i < coches.length; i++) {
  console.log(coches[i]);
}

for (let k = 0; ; k++) {
  if (k > 3) break;
  console.log("k =", k);
}

let j = 0;
for (; j < 3; ) {
  console.log("j =", j);
  j++;
}
```

```
Citroen
Ford
k = 0
k = 1
k = 2
k = 3
j = 0
j = 1
j = 2
```

- Se puede omitir la inicialización si la variable ya existe, la condición si hay un `break`, o el incremento si se hace dentro.
- `i` se declara fuera, así que después del bucle conserva el valor 4. Con `let` dentro del `for`, moriría al terminar.
- **Trampa:** `for (let k = 0; ; k++)` sin `break` es un bucle infinito. Y `k = 3` sí se imprime: el corte es `k > 3`.

Ejemplo 4 — `for...in` sobre un array

```
Array.prototype.nuevoMetodo = "basura del prototipo";
const colores = ["rojo", "azul"];

for (const indice in colores) {
  console.log(`Índice: ${indice} (${typeof indice}) -> ${colores[in
dice]}`);
}
console.log("---");
for (const color of colores) {
  console.log(color);
}
```

```
Índice: 0 (string) -> rojo
Índice: 1 (string) -> azul
Índice: nuevoMetodo (string) -> basura del prototipo
---
rojo
azul
```

- `for...in` recorre claves, y en un array las claves son los índices, en forma de cadena.
- También recorre las propiedades añadidas al prototipo, que es la razón de no usarlo con arrays.
- `for...of` recorre valores, no ve el prototipo y es lo correcto para arrays y cadenas.
- **Trampa:** el índice de `for...in` es `"0"`, no `0`. Cualquier suma se convierte en concatenación.

Ejemplo 5 — La trampa del índice en cadena

```
const letras = ["a", "b", "c"];
for (const i in letras) {
  console.log(i + 1);
}
console.log("---");
for (let i = 0; i < letras.length; i++) {
  console.log(i + 1);
}
```

```
01
11
21
---
1
2
3
```

- En el primero, `i` es `"0"` y `i + 1` concatena: `"01"`.
- En el segundo, `i` es número y suma de verdad.
- **Trampa:** si la pregunta es "qué imprime", la diferencia entre `1` y `"01"` es toda la respuesta. Y `"21"` no es veintiuno, es el texto de la clave `"2"` seguido de un 1.

Ejemplo 6 — Arrays dispersos: `forEach` salta los huecos

```
const disperso = [1, , 3];
console.log("length:", disperso.length);
for (let i = 0; i < disperso.length; i++) {
  console.log("for:", i, disperso[i]);
}
disperso.forEach((valor, indice) => console.log("forEach:", indice,
 valor));
for (const valor of disperso) {
  console.log("for...of:", valor);
}
```

```
length: 3
for: 0 1
for: 1 undefined
for: 2 3
forEach: 0 1
forEach: 2 3
for...of: 1
for...of: undefined
for...of: 3
```

- `[1, , 3]` tiene longitud 3 pero el índice 1 no existe: es un hueco, no un `undefined` almacenado.
- El `for` clásico visita el hueco y devuelve `undefined`; `forEach` lo salta; `for...of` lo visita y también da `undefined`.
- **Trampa:** `length` cuenta huecos. `[1, , 3].length` es 3, y un `filter` sobre ese array ni siquiera llama al callback para el hueco.

Ejemplo 7 — `forEach` no admite `break`

```
const numeros = [1, 2, 3, 4];
numeros.forEach((n) => {
  if (n === 3) break;
  console.log(n);
});
```

```
SyntaxError: Illegal break statement
```

- El `break` está dentro de una función flecha, que no es un bucle: el motor lo rechaza al analizar el archivo.
- Es un error de sintaxis, así que no se ejecuta nada, ni siquiera el primer `console.log`.
- **Trampa:** `return` dentro de `forEach` no corta el bucle, solo salta esa iteración; el array `[1,2,3]` imprimiría 1 y 3. Para cortar de verdad: `for...of` con `break`, o `some()` y `every()`.

Ejemplo 8 — `break` y `continue`

```
for (let i = 1; i <= 10; i++) {
  if (i === 6) {
    console.log("Corte de emergencia en i = 6");
    break;
  }
  console.log("Vuelta", i);
}

for (let n = 1; n <= 6; n++) {
```

```
  if (n % 2 === 0) continue;
  console.log("Impar:", n);
}
```

```
Vuelta 1
Vuelta 2
Vuelta 3
Vuelta 4
Vuelta 5
Corte de emergencia en i = 6
Impar: 1
Impar: 3
Impar: 5
```

- `break` abandona el bucle entero; `continue` salta el resto de esa vuelta y pasa a la siguiente.
- En un `for`, `continue` ejecuta el incremento antes de volver a la condición, así que no se queda colgado.
- **Trampa:** un `break` dentro de un `switch` solo sale del `switch`. Si el `switch` está dentro de un bucle, el bucle sigue; para salir de los dos hace falta una etiqueta.

Ejemplo 9 — Etiquetas en bucles

```
bucleMatriz: for (let fila = 1; fila <= 3; fila++) {
  for (let columna = 1; columna <= 3; columna++) {
    if (fila === 2 && columna === 2) {
      console.log("Coincidencia en 2,2: abortando la matriz");
      break bucleMatriz;
    }
    console.log(`Celda ${fila},${columna}`);
  }
}
console.log("Fuera del bucle etiquetado");
```

```
Celda 1,1
Celda 1,2
Celda 1,3
Celda 2,1
Coincidencia en 2,2: abortando la matriz
Fuera del bucle etiquetado
```

- `break bucleMatriz` sale de los dos bucles a la vez. Sin la etiqueta, solo saldría del interno y seguiría con la fila 3.
- La etiqueta es un nombre seguido de dos puntos delante del bucle; no afecta a la ejecución por sí misma.
- **Trampa:** también existe `continue bucleMatriz`, que salta a la siguiente vuelta del bucle exterior. Con dos bucles anidados es fácil confundirlos.

Ejemplo 10 — Etiqueta sobre un bloque

```
const coches = ["BMW", "Volvo", "Saab", "Ford"];
let texto = "";

list: {
  texto += coches[0] + " ";
  texto += coches[1] + " ";
  break list;
  texto += coches[2] + " ";
  texto += coches[3] + " ";
```

```
}

console.log(texto);
```

```
BMW Volvo
```

- Una etiqueta puede marcar cualquier bloque, no solo un bucle. `break list` sale del bloque y continúa después.
- Las dos últimas líneas no se ejecutan nunca. Queda un espacio final en la cadena, que la consola no muestra.
- **Trampa:** una etiqueta no es un `goto`: solo se puede romper hacia delante y, en bucles, continuar. No se puede saltar a una etiqueta.

Ejemplo 11 — El bug del `splice`

```
const numeros = [2, 4, 6, 8, 9];

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    numeros.splice(i, 1);
  }
}
console.log(numeros);
console.log(numeros.length);
```

```
[4, 8, 9]
3
```

- Traza: en `i = 0` se borra el 2 y el array pasa a `[4, 6, 8, 9]`; el 4 se desplaza al índice 0, pero `i` avanza a 1.
- En `i = 1` se borra el 6, no el 4: `[4, 8, 9]`. En `i = 2` está el 9, que es impar. En `i = 3` ya no hay nada.
- El resultado no es "los impares": queda el 4 y el 8. El bug no da error, deja basura.
- **Trampa:** el comentario que trae el enunciado dice `[4, 6, 9]`, pero la ejecución real da `[4, 8, 9]`. Comprobado con Node; la traza manda.

Ejemplo 12 — El arreglo: recorrer al revés o `filter`

```
const numeros = [2, 4, 6, 8, 9];

for (let i = numeros.length - 1; i >= 0; i--) {
  if (numeros[i] % 2 === 0) {
    numeros.splice(i, 1);
  }
}
console.log(numeros);

const otros = [2, 4, 6, 8, 9];
console.log(otros.filter((n) => n % 2 !== 0));
```

```
[9]
[9]
```

- De atrás adelante, el `splice` no desplaza ningún índice que aún quede por visitar: el bucle es correcto.
- `filter` devuelve un array nuevo con los impares y no modifica el original. Es la opción declarativa.

- **Trampa:** si el ejercicio pide modificar el array original, `filter` no sirve tal cual: hay que reasignar (`numeros = numeros.filter(...)`) o usar el recorrido decreciente.

Ejemplo 13 — El rombo

```
const filas = 4;

for (let i = 1; i <= filas; i++) {
  let linea = "";
  for (let j = 1; j <= filas - i; j++) linea += " ";
  for (let j = 1; j <= 2 * i - 1; j++) linea += "*";
  console.log(linea);
}

for (let i = filas - 1; i >= 1; i--) {
  let linea = "";
  for (let j = 1; j <= filas - i; j++) linea += " ";
  for (let j = 1; j <= 2 * i - 1; j++) linea += "*";
  console.log(linea);
}
```

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

- Dos bucles: el primero sube de 1 a 4 y el segundo baja de 3 a 1. La mitad inferior es la superior al revés, sin repetir la línea central.
- En cada fila, los espacios son `filas - i` y los asteriscos son `2 * i - 1`: la fila central tiene 7.
- **Trampa:** con `filas = 4` salen 7 líneas, no 8. El total es siempre `2 * filas - 1`, y las líneas impares de asteriscos (1, 3, 5, 7, 5, 3, 1).

Ejemplo 14 — Validación con `prompt` y `do...while`

```
const respuestas = ["abc", "12", "1234"];
let turno = 0;
let codigo;

do {
  codigo = respuestas[turno++];
  if (codigo === null) break;
  codigo = codigo.trim();
} while (Number.isNaN(Number(codigo)) || codigo.length !== 4);

console.log("Intentos:", turno);
console.log("Código aceptado:", codigo);
```

```
Intentos: 3
Código aceptado: 1234
```

- En el navegador, la primera línea sería `codigo = prompt("Código de 4 dígitos:");`. Aquí se sustituye por un array para poder comprobar la salida sin teclado.
- `prompt` devuelve siempre una cadena, o `null` si se cancela. Por eso hay que convertir con `Number` para saber si es numérico, y comprobar la longitud aparte.
- La condición repite mientras el valor no sea numérico (`Number.isNaN`) o no mida 4.

`"abc"` y `"12"` fallan; `"1234"` sale.

- **Trampa:** `Number("")` es 0, que no es `NaN`, así que una entrada vacía pasaría la primera prueba. Sin el `length !== 4` se aceptaría. Y `prompt` puede devolver `null`: sin el `break`, `null.trim()` lanzaría `TypeError`.

Qué bucle usar

Bucle	Cuándo usarlo	Iteraciones mínimas
`while`	No se sabe cuántas vueltas; condición al principio	0
`do...while`	Al menos una vuelta: menús, validación de entrada	1
`for`	Número de vueltas conocido o contador explícito	0
`for...of`	Valores de un array o una cadena	Una por elemento
`for...in`	Claves de un objeto	Una por propiedad enumerable
`forEach`	Acción por elemento sin cortar el recorrido	Salta los huecos

Banco de 20 preguntas tipo test

1. ¿Qué imprime `console.log(typeof null)`? a) `"null"` · b) `"object"` · c) `"undefined"` · d) `SyntaxError`. — **b)**. Anomalía histórica: `null` se representaba con la etiqueta de objeto y se mantuvo por compatibilidad.

2. `let x = 5; x = "5"; console.log(typeof x);` a) `"number"` · b) `"string"` · c) `"boolean"` · d) `TypeError`. — **b)**. Tipado dinámico: el tipo lo decide el valor actual, no la declaración.

3. `console.log(0 == "", 0 === "")` a) `true true` · b) `false false` · c) `true false` · d) `false true`. — **c)**. `==` convierte la cadena vacía en 0; `===` compara también el tipo.

4. `console.log(null ?? 10, 0 ?? 10, "" ?? 10)` a) `10 10 10` · b) `10 0 ""` · c) `10 0 10` · d) `null 0 ""`. — **b)**. `??` solo sustituye `null` y `undefined`.

5. Con `let i = 2; console.log(i++ * ++i, i);` a) `8 4` · b) `9 4` · c) `6 3` · d) `8 5`. — **a)**. El postincremento aporta 2, el preincremento 4, y `i` acaba en 4.

6. `console.log(typeof c); let c = 10;` a) `"undefined"` · b) `"number"` · c) `ReferenceError: c is not defined` · d) `ReferenceError: Cannot access 'c' before initialization`. — **d)**. TDZ: `typeof` no protege a una variable declarada antes de su línea.

7. `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));` a) `0 1 2` · b) `3 3 3` · c) `0 0 0` · d) `1 2 3`. — **b)**. `var` comparte una sola variable y los temporizadores se ejecutan cuando el bucle ya terminó.

8. `function f() { let c = 0; return () => ++c; } const g = f(); g(); g(); console.log(g());` a) `1` · b) `2` · c) `3` · d) `0`. — **c)**. El closure conserva `c` entre llamadas.

9. `console.log("10" < "9", "10" < 9)` a) `true true` · b) `false false` · c) `true false` · d) `false true`. — **c)**. Dos cadenas se comparan por código de carácter; con un número, la cadena se convierte.

10. `console.log([] == false, [] === false)` a) `true true` · b) `true false` · c) `false false` · d) `false true`. — **b)**. `[]` se convierte en `""` y luego en 0; con `===` son tipos

distintos.

11. `console.log(NaN === NaN, Object.is(NaN, NaN))` a) `true true` · b) `false false` · c) `false true` · d) `true false`. — **c)**. `NaN` no es igual a sí mismo con `===`; `Object.is` lo trata como igual.

12. ¿Qué imprime `switch (2)` con `case 1: "Uno"; case 2: "Dos"; case 3: "Tres";` y sin ningún `break`? a) `Dos` · b) `Dos Tres` · c) `Uno Dos Tres` · d) Nada. — **b)**. Empieza en el `case 2` y cae hacia abajo.

13. `switch (99)` con `default` en primer lugar y dos `case` detrás, sin `break`. a) Solo el default · b) El default y los dos case · c) Nada · d) `SyntaxError`. — **b)**. El `default` puede ir en medio y también arrastra el fall-through.

14. ¿Qué reduce una cláusula de guarda? a) La complejidad ciclomática · b) El anidamiento y los `else` · c) El número de `return` · d) El tiempo de ejecución. — **b)**. Baja la profundidad de sangría y elimina los `else` tras un `return`; los puntos de decisión siguen ahí.

15. `const n = [2,4,6,8,9];` borrando los pares con `splice(i, 1)` en un `for` ascendente, ¿qué queda? a) `[9]` · b) `[4, 8, 9]` · c) `[4, 6, 9]` · d) `[2, 4, 6, 8]`. — **b)**. El desplazamiento del array hace que el bucle se salte elementos.

16. `for (const i in ["a","b","c"]) console.log(i + 1);` a) `1 2 3` · b) `01 11 21` · c) `a1 b1 c1` · d) `0 1 2`. — **b)**. `for...in` da las claves como cadenas y `+` concatena.

17. Un `break` dentro del callback de `forEach`… a) corta el bucle · b) salta la iteración · c) lanza `SyntaxError: Illegal break statement` · d) no hace nada. — **c)**. El callback no es un bucle y el error es de sintaxis, antes de ejecutar.

18. `let x = false; do { console.log("vez"); } while (x);` a) Nada · b) Una vez · c) Infinitas · d) `SyntaxError`. — **b)**. La condición se evalúa después del cuerpo.

19. Con `filas = 4`, el rombo del tema imprime… a) 4 líneas · b) 8 líneas · c) 7 líneas y la central con 7 asteriscos · d) 7 líneas y la central con 4 asteriscos. — **c)**. Son `2 * filas - 1` líneas y `2 * i - 1` asteriscos en la fila central.

20. ¿Cuál de estas afirmaciones sobre `defer` es correcta? a) Funciona también en scripts embebidos · b) Ejecuta el script en cuanto termina de descargarlo · c) Espera a que el HTML esté parseado y respeta el orden · d) No descarga el script hasta que se necesita. — **c)**. `async` es el que se ejecuta al terminar la descarga y no respeta el orden; ninguno de los dos actúa en scripts embebidos.

Chuleta: una línea por regla

- El punto y coma se escribe siempre: el ASI adivina y a veces adivina mal.
- `return` y la llave del objeto van en la misma línea, o la función devuelve `undefined`.
- Los identificadores empiezan por letra, `$` o `_`; nunca por número ni por palabra reservada.
- JavaScript distingue mayúsculas: `totalVentas` y `TotalVentas` son dos variables.
- `var` es de función, `let` y `const` son de bloque.
- `const` sin valor es `SyntaxError`; reasignarla es `TypeError`.
- `const` no congela el contenido: congela el nombre.
- Antes de su línea, `var` vale `undefined`; `let` y `const` están en TDZ y lanzan

`ReferenceError`.

- `typeof` no lanza con una variable no declarada, pero sí con una en TDZ.
- Las declaraciones de función se elevan completas; las expresiones y las flechas, no.
- `"use strict"` convierte la global accidental en `ReferenceError`.
- Un closure es una función más el ámbito donde nació; cada llamada al creador abre un ámbito nuevo.
- `var` en un bucle con `setTimeout` da el último valor repetido; `let` da uno por iteración.
- `typeof null` es `"object"` y `typeof []` también: para arrays, `Array.isArray`.
- `typeof NaN` es `"number"`.
- `+` concatena si hay una cadena; `-`, `*`, `/` y `%` siempre convierten a número.
- `[] + []` es la cadena vacía; `[] + {}` es `"[object Object]"`.
- Los falsy son ocho: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`.
- `[]` y `{}` son truthy: `if (arr)` no comprueba si está vacío.
- `null == undefined` es `true`; `null == 0` es `false`.
- `NaN` no es igual a nada, ni a sí mismo: `Number.isNaN`.
- `Object.is` distingue `0` de `-0` y hace `NaN` igual a `NaN`.
- `??` solo sustituye `null` y `undefined`; `||` sustituye cualquier falsy.
- `??` no se mezcla con `||` ni con `&&` sin paréntesis.
- `switch` compara con `===`: `"1"` no coincide con `1`.
- Sin `break`, el `switch` sigue ejecutando hacia abajo.
- El `default` puede estar en medio y también arrastra el fall-through.
- El ternario devuelve un valor; para tres ramas o más, `if` o `switch`.
- Una guarda valida y sale: baja el anidamiento y elimina los `else`.
- El orden de las guardas importa: primero que el objeto exista, después sus propiedades.
- `console.assert` no detiene el programa y no escribe nada si la condición es verdadera.
- `while` puede dar 0 vueltas; `do...while` da 1 como mínimo.
- `for` admite omitir inicialización, condición e incremento, pero sin condición hace falta `break`.
- `for...in` da claves en forma de cadena e incluye el prototipo: no se usa con arrays.
- `for...of` da valores y no ve el prototipo.
- `forEach` salta los huecos y no admite `break` ni `continue`.
- `return` dentro de `forEach` solo salta esa iteración.
- `break` sale del bucle; `continue` salta a la siguiente vuelta.
- Un `break` solo sale del `switch` más cercano, no del bucle que lo contiene.
- `break etiqueta` sale de varios bucles a la vez.
- Borrar con `splice` dentro de un `for` ascendente se salta elementos: recorre al revés o usa `filter`.
- Un array disperso tiene `length` contando huecos, aunque falten índices.
- `prompt` devuelve cadena o `null`; `Number("")` es 0, así que la longitud se comprueba aparte.